School of Computer Science and Engineering

(Computer Science & Engineering)

Faculty of Engineering & Technology

Jain Global Campus, Kanakapura Taluk - 562112

Ramanagara District, Karnataka, India

2026-2027

(VI Semester)

A Project Report on

"AI in Hospital Inventory Management"

Submitted in partial fulfilment for the award of the degree of

Bachelor of Technology

in

COMPUTER SCIENCE AND ENGINEERING

(Artificial Intelligence And Machine Learning)

Submitted by

Tenzin Kunga – 23BTRCL088, Saumitra Purkayastha – 23BTRCL113, Saunak Mahapatra – 23BTRCL077

Under the guidance of

Dr. S Kanithan

Assistant Professor – AIML

Jain (Deemed-to-be) University, Bengaluru, Karnataka.

CERTIFICATE

This is to certify that the project work titled **"AI in Hospital Inventory Management"** is carried out by **Tenzin Kunga (23BTRCL088), Saumitra Purkayastha (23BTRCL113), Saunak Mahapatra (23BTRCL077)** bonafide student(s) of Bachelor of Technology at the School of Engineering & Technology, Faculty of Engineering & Technology, JAIN (Deemed-to-be University), Bangalore in partial fulfillment for the award of degree in Bachelor of Technology in Computer Science and Engineering, during the year **2026-2027**.

Dr. S Kanithan	**Dr. V Chandrashekar**	**Dr. Geetha G**
Assistant Professor, AIML School of Computer Science & Engineering JAIN (Deemed-to-be University) Date: 21 April 2026	Computer Science and Engineering School of Computer Science & Engineering JAIN (Deemed-to-be University) Date: 21 April 2026	Director, School of Computer Science & Engineering Faculty of Engineering & Technology JAIN (Deemed-to-be University) Date: 21 April 2026

Name of the Examiner Signature of Examiner

DECLARATION

We, **Tenzin Kunga (23BTRCL088), Saumitra Purkayastha (23BTRCL113), Saunak Mahapatra (23BTRCL077)** students of VI Semester B.Tech in **Computer Science and Engineering (Artificial Intelligence and Machine Learning)**, at School of Engineering & Technology, Faculty of Engineering & Technology, **JAIN (Deemed-to-be University)**, hereby declare that the project work titled **"AI in Hospital Inventory Management"** has been carried out by us and submitted in partial fulfilment for the award of degree in **Bachelor of Technology in Computer Science and Engineering** during the academic year **2026-2027**. Further, the matter presented in the work has not been submitted previously by anybody for the award of any degree or any diploma to any other University, to the best of our knowledge and faith.

Name	**USN**	**Signature**
Tenzin Kunga	23BTRCL088	
Saumitra Purkayastha	23BTRCL113	
Saunak Mahapatra	23BTRCL077	

ACKNOWLEDGEMENT

It is a great pleasure for us to acknowledge the assistance and support of many individuals who have been responsible for the successful completion of this project work.

First, we take this opportunity to express our sincere gratitude to the Faculty of Engineering & Technology, JAIN (Deemed-to-be University) for providing us with a great opportunity to pursue our bachelor's degree in this institution.

We are deeply thankful to several individuals whose invaluable contributions have made this project a reality. We wish to extend our heartfelt gratitude to **Dr. Chandraj Roy Chand, Chancellor**, for his tireless commitment to fostering excellence in teaching and research. We are also profoundly grateful to the honourable **Vice Chancellor, Dr. Raj Singh**, and **Dr. Dinesh Nilkant, Pro Vice Chancellor**, for their unwavering support. Furthermore, we would like to express our sincere thanks to **Dr. Jitendra Kumar Mishra, Registrar**, whose guidance has imparted invaluable qualities and skills.

We extend our sincere gratitude to **Dr. Hariprasad S A, Director** of the Faculty of Engineering & Technology, and **Dr. Geetha G, Director** of the School of Computer Science & Engineering, for their constant encouragement and expert advice. We also appreciate **Dr. Krishnan Batri, Deputy Director (Course and Delivery)**, and **Dr. V. Vivek, Deputy Director (Students & Industry Relations)**, for their invaluable contributions throughout this project.

It is a matter of immense pleasure to express our sincere thanks to **Dr. V Chandrashekhar, Program Head of Computer Science and Engineering (AIML)**, for providing the right academic guidance. We would like to thank our guide **Dr. S Kanithan, Assistant Professor, Jain (Deemed-to-be) University**, for sparing his valuable time to extend help in every step of our work, which paved the way for smooth progress and fruitful culmination of the project.

Signature of Student(s)

Tenzin Kunga – 23BTRCL088

Saumitra Purkayastha – 23BTRCL113

Saunak Mahapatra – 23BTRCL077

ABSTRACT

Hospital inventory management is one of the most critical and complex operations in the healthcare sector. Medical supplies, pharmaceuticals, and equipment are often perishable, expensive, and subject to highly unpredictable demand influenced by emergencies, seasonal outbreaks, surgical schedules, and patient acuity. Traditional inventory systems — frequently reliant on manual processes, spreadsheets, and static PAR levels — struggle to address these challenges efficiently.

AHIMP (AI-Powered Hospital Inventory Management Platform) is a comprehensive, intelligent solution that integrates advanced machine learning, explainable AI, and autonomous AI agents to modernize hospital inventory management. The platform employs an ensemble of state-of-the-art algorithms including LightGBM, CatBoost, LSTM/GRU recurrent networks, and weighted voting to deliver highly accurate demand forecasting ($R^2 \geq 0.97$), stockout risk prediction, and expiry risk assessment across a dataset of approximately 7.3 million synthetic consumption records.

Transparency and clinical trust are ensured through SHAP (SHapley Additive exPlanations) and LIME (Local Interpretable Model-agnostic Explanations) integration, providing interpretable, feature-level justifications for every AI recommendation. The autonomous CrewAI-based agents — running locally with Ollama/llama3 for privacy and cost efficiency — automate the entire procurement lifecycle: intelligent data ingestion with anomaly quarantine, composite supplier scoring (incorporating reliability, price, delivery, and NLP sentiment analysis), optimized purchase order generation, a multi-tier approval workflow, and real-time delivery tracking with delay alerts.

The system is built on a modern technology stack comprising a Next.js frontend, FastAPI backend, PostgreSQL database, and Docker Compose deployment. Testing coverage spans unit tests, integration tests, API tests, and model benchmarking, with all critical paths validated. Key operational outcomes include a real-time dashboard tracking 680 items across 12 hospital departments, proactive expiry and low-stock alerts, automated purchase order workflows, and department-level budget utilization reporting — all contributing to measurable reductions in stockouts, expiry waste, and procurement overhead.

Keywords: Hospital Inventory Management, LightGBM, CatBoost, LSTM/GRU, Ensemble Learning, SHAP, LIME, CrewAI, Autonomous Agents, Demand Forecasting, Explainable AI, FastAPI, Next.js, Docker.

LIST OF FIGURES

TABLE OF CONTENTS

1. INTRODUCTION

Hospital inventory management is one of the most complex and critical operations in healthcare. Medical supplies, pharmaceuticals, and equipment are often perishable, expensive, and subject to highly unpredictable demand influenced by emergencies, seasonal outbreaks, surgical schedules, patient acuity, and external events such as pandemics. Unlike traditional retail or manufacturing, hospitals must maintain continuous availability of life-saving items while minimizing waste from expiry, overstocking, and inefficient procurement.

Traditional inventory systems — frequently reliant on manual processes, spreadsheets, static PAR (Periodic Automatic Replenishment) levels, or basic ERP modules — struggle with these challenges. Common issues include frequent stockouts that delay critical procedures and compromise patient care, significant expiry-related waste leading to financial losses and environmental burden, poor demand forecasting due to volatile usage patterns, data quality problems from manual entry, and slow, reactive supply chain processes. These inefficiencies result in higher operational costs, tied-up capital, increased emergency ordering, and unnecessary administrative burden on clinical staff.

1.1 Motivation

The motivation behind AHIMP stems from the urgent need to transform hospital inventory management from a reactive, error-prone process into a proactive, intelligent, and transparent one. By leveraging advances in machine learning, explainable AI, and autonomous AI agents, the platform aims to address core pain points: inaccurate forecasting, stockouts, expiry waste, suboptimal supplier selection, and lack of trust in automated decisions.

Healthcare professionals require systems that not only predict demand accurately but also explain why a prediction or recommendation is made — essential for regulatory compliance, clinical trust, and adoption. Additionally, automating repetitive tasks (data ingestion, supplier scoring, purchase order generation, and delivery tracking) can free pharmacists and procurement teams to focus on patient-centric activities. Running AI agents locally with privacy-focused models (such as Ollama/llama3) further addresses concerns around data security, cost, and cloud dependency in sensitive healthcare environments.

1.2 Aim and Objectives

Aim:

The primary aim of AHIMP is to develop an AI-powered, end-to-end hospital inventory management platform that integrates advanced predictive analytics, explainable AI, and autonomous agents to enable accurate demand forecasting, proactive risk mitigation, waste reduction, and intelligent automated procurement — all within a modern, user-friendly interface that supports real-time decision-making and regulatory compliance.

Objectives:

- Build and ensemble advanced ML models (LightGBM, CatBoost, LSTM/GRU, Linear Regression) capable of handling large-scale historical consumption data (~7.3 million records) to predict future demand, stockout risk, and expiry risk with high accuracy.
- Incorporate Explainable AI: Integrate SHAP and LIME techniques to provide transparent, interpretable explanations for model predictions and recommendations, enabling clinicians and pharmacists to understand and trust AI outputs.
- Implement autonomous AI agents: Create CrewAI-based agents (running locally with Ollama/llama3) for intelligent data ingestion & validation, supplier scoring (including NLP sentiment analysis), stockout prevention, automated purchase order generation, approval workflows, and delivery tracking with delay alerts.
- Design robust validation, anomaly detection (Isolation Forest), and synthetic data generation pipelines to maintain clean, reliable data while supporting hospital-scale volumes.
- Develop a modern Next.js frontend with interactive visualizations, agent monitoring, approval queues, and near real-time updates to support operational visibility and decision-making.

1.3 Project Scope

- Real-time tracking and management of medical supplies, drugs, and equipment.
- AI-powered demand forecasting, stockout risk prediction, and expiry risk assessment.
- Autonomous AI agents (using CrewAI + local Ollama/llama3) for anomaly detection, supplier scoring, PO generation, and delivery tracking.
- Modern web dashboard with visualizations, alerts, agent monitoring, and manual approval workflows.
- Containerized deployment using Docker Compose for consistent, reproducible environments.

1.4 User Classes and Characteristics

Pharmacists / Inventory Managers (Primary Users): Manage daily stock, review forecasts, approve POs, monitor alerts. Tech-savvy, need quick actionable insights.

Procurement / Supply Chain Staff: Handle supplier scoring, PO creation, delivery tracking. Require audit trails and optimization tools.

Hospital Administrators / Clinicians: View high-level dashboards, risk alerts, and explainable reports for oversight and compliance.

Data Entry / IT Staff: Perform data ingestion, review quarantined anomalies, and manage system configuration.

Developers / Maintainers: Access APIs, models, and logs for maintenance and extension.

1.5 Assumptions and Dependencies

Assumptions:

- Historical consumption data (or synthetic data) is available for model training.
- Users have access to a modern web browser.
- Local Ollama instance with the llama3 model is available for AI agents.
- PostgreSQL database is used as the primary data store.
- The system will start with synthetic data for testing and can ingest real CSV/API data later.

Dependencies:

- External: Docker & Docker Compose for deployment; Ollama for agent reasoning.
- Internal: Trained ML models (LightGBM, CatBoost, ensemble, etc.)
- Third-party libraries: FastAPI, SQLAlchemy, scikit-learn, SHAP, CrewAI, Next.js ecosystem (see requirements.txt and package.json).
- Training time on first run (5–10+ minutes) due to large synthetic dataset.

2. LITERATURE SURVEY

Artificial intelligence is increasingly applied in healthcare to support disease detection, chronic condition management, drug discovery, and supply chain operations. Most current AI systems are narrow, performing specific tasks through machine learning that identifies patterns in data rather than following fixed rules. While AI holds promise for addressing challenges like ageing populations and resource constraints, its effectiveness is often limited by poor data quality, inherent biases in training datasets, and its inability to replicate human qualities such as empathy or complex ethical judgment.

Several studies demonstrate the practical value of AI in healthcare supply chains, which face high product variety, strict regulations, and time-sensitive demand that can directly affect patient care. One large-scale implementation analyzed over 24 months of data covering thousands of SKUs and hundreds of hospitals. It integrated demand forecasting, service-level modelling, and multi-echelon inventory optimization, resulting in a 14.78 percentage-point improvement in True Service Level, elimination of nearly 1,900 surgical kit shortages, a substantial reduction in global inventory worth over $144 million, and a 55.3% gain in forecast accuracy.

Researchers classify AI by capability (narrow/weak, general/strong, and theoretical superintelligent), functionality (reactive, limited memory, theory of mind, self-aware), and methodology, with machine learning, deep learning, natural language processing, computer vision, and robotics being most relevant to inventory tasks. In healthcare settings, these tools improve demand forecasting, real-time tracking, predictive maintenance, and warehouse automation. They enable proactive rather than reactive inventory control, helping reduce waste, lower carrying costs, ensure timely availability of critical supplies, and support regulatory compliance while improving overall patient outcomes.

Complementary technologies such as RFID further strengthen these systems. Experimental deployments have shown RFID achieving high tag-reading speeds, cutting equipment retrieval time by about 30%, reducing stock shortages and overstocking, raising inventory accuracy to near 98%, and boosting staff productivity by allowing more time for direct patient care. When combined with AI and IoT, such technologies create smarter, interconnected environments for better visibility and decision-making.

Additional work on hospital pharmacy automation highlights robotic dispensing and AI-enabled clinical support as tools that reduce medication errors, save time, and enhance safety. However, challenges persist around data quality, integration with legacy systems, high implementation costs, staff training needs, and ensuring transparency and accountability in AI models. Emerging opportunities include digital twins, real-time analytics, and phased adoption strategies that emphasize data governance and stakeholder engagement.

Research on reinforcement learning further broadens the solution space for inventory optimization. Deep reinforcement learning (DRL) methods have demonstrated capability in multi-echelon settings where demand is stochastic and lead times vary. Studies in retail and healthcare supply chains show DRL-based agents can outperform classical approaches (EOQ, base-stock policies) in volatile environments by learning adaptive replenishment policies directly from historical data.

3. FUNCTIONAL & NON-FUNCTIONAL REQUIREMENTS

3.1 Functional Requirements

The system shall provide the following core functionalities:

Inventory Management

- CRUD operations for items (drugs, supplies, equipment) with details like category, supplier, expiry date, reorder point, and current stock.
- Real-time stock level tracking and history of movements.

AI-Powered Forecasting & Risk Prediction

- Demand forecasting for 14+ days using ensemble models (LightGBM + CatBoost + LSTM/GRU + others).
- Stockout risk and expiry risk prediction for all inventory items.
- Anomaly detection in consumption patterns using Isolation Forest.

Explainable AI

- Generate SHAP/LIME explanations for individual predictions and global feature importance.
- Provide interpretable JSON payloads for frontend visualization.

AI Agents

- Data Ingestion Agent: Validate & ingest CSV/API data, detect duplicates/anomalies, quarantine suspicious records.
- Supply Chain Agent: Monitor stockout risk, score suppliers, generate optimized purchase orders, and trigger approvals.
- Delivery Tracking Agent: Monitor PO status, send delay alerts, and maintain audit logs.

Procurement & Approval Workflow

- Automated PO generation with optimal quantity and supplier selection.
- Manual approval queue for high-value or critical orders with scoring breakdown and comments.
- Timeout-based auto-approval rules for low-value, high-reliability orders.

Dashboard & Reporting

- Interactive visualizations: demand trends, stock status, alerts, expiry calendar, cost savings.
- Agent dashboard: ingestion status, at-risk items, supplier recommendations, PO tracker, execution logs.

3.2 Non-Functional Requirements

Performance:

- API response time < 500ms for 95th percentile of requests.
- Forecasting inference for all items < 30 seconds.
- Agent cycle time < 30 seconds for SLA compliance.

Scalability:

- Support up to 1,000 inventory items and 50+ concurrent users.
- Handle dataset growth to 10M+ records without performance degradation.

Reliability:

- 99% uptime for critical inventory and dashboard APIs.
- Graceful fallback to best single model if ensemble component fails.

Security:

- Role-based access control for all API endpoints.
- Local model inference (Ollama) to prevent data leakage to external AI services.

Maintainability:

- Modular, well-documented codebase with comprehensive unit and integration tests.
- Docker Compose for reproducible deployment and easy updates.

4. SYSTEM AND HARDWARE REQUIREMENTS

4.1 Hardware Requirements

Component	Minimum Requirement	Recommended
Processor	Quad-core 2.0 GHz	8-core 3.0 GHz (Intel i7/AMD Ryzen 7)
RAM	8 GB	16–32 GB (for ML training)
Storage	20 GB SSD	50+ GB NVMe SSD
GPU (optional)	Not required	NVIDIA GPU (CUDA) for LSTM training
Network	Broadband internet	Gigabit LAN for hospital integration

4.2 Software Requirements

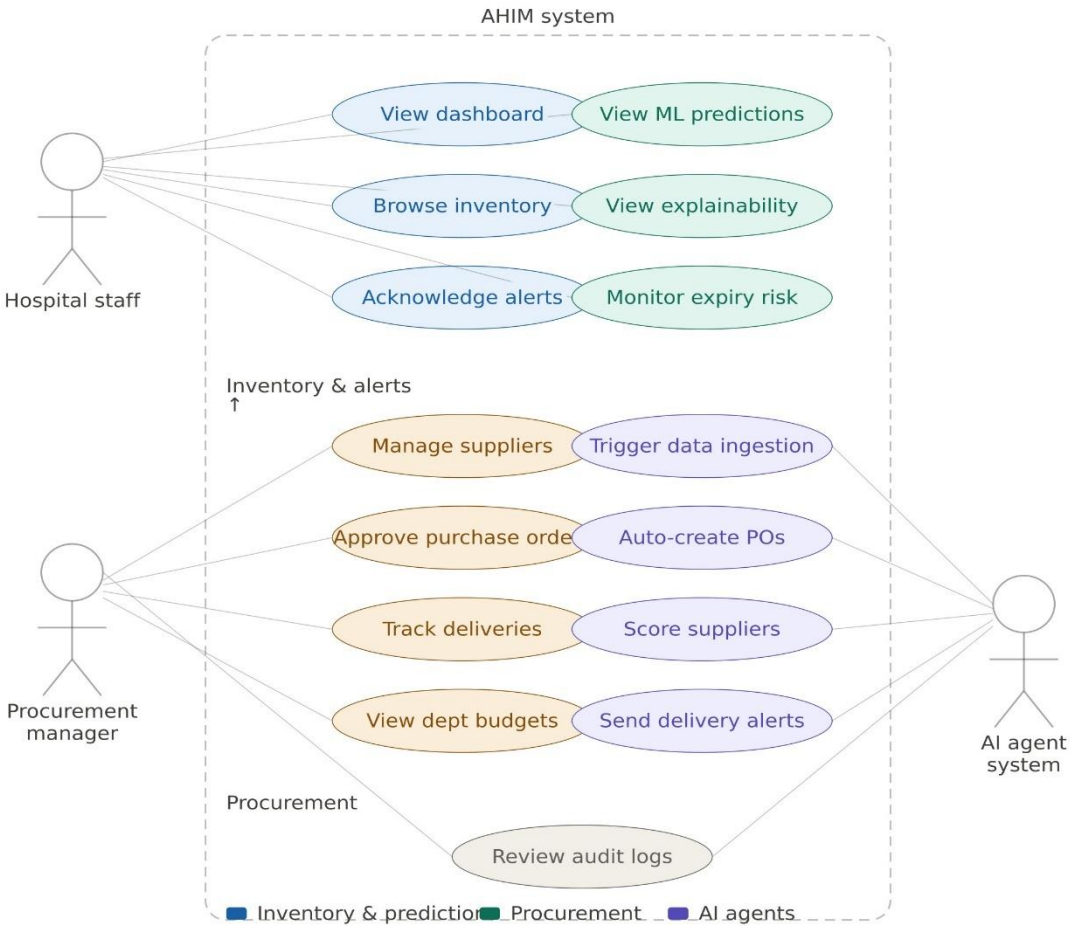
Category	Technology / Version	Purpose
Operating System	Ubuntu 22.04 / Windows 10+	Host environment
Container Runtime	Docker 24+, Docker Compose v2	Deployment & orchestration
Backend Language	Python 3.11+	ML models & API server
Backend Framework	FastAPI 0.110+	REST API endpoints
Database	PostgreSQL 15+	Primary data store
ORM	SQLAlchemy 2.0+	Database abstraction
ML Libraries	LightGBM, CatBoost, TensorFlow/Keras	Model training & inference
Explainability	SHAP, LIME	AI transparency
Agent Framework	CrewAI + Ollama/llama3	Autonomous agents
Frontend	Next.js 14+, React 18, TypeScript	Web dashboard
Charts	Recharts / D3.js	Data visualizations
Testing	pytest, unittest.mock	Test suite

5. DESIGN AND MODELLING OF SYSTEM

Unified Modelling Language (UML) is analogous to the blueprints used in other fields and consists of different types of diagrams. In the aggregate, UML diagrams describe the boundary, structure, and behaviour of the system and the objects within it. The following UML diagrams have been designed for the AHIMP project:

5.1 Use Case Diagram

Use case diagrams are behaviour diagrams used to describe a set of actions that some system or systems should or can perform in collaboration with one or more external users. The AHIMP use case diagram models the interactions between five primary actors — Pharmacist/Inventory Manager, Procurement Staff, Hospital Administrator, Data Entry/IT Staff, and AI Agent System — and the core system use cases including Manage Inventory, View Forecasts, Approve Purchase Orders, Monitor Alerts, Configure Agents, and Generate Reports.

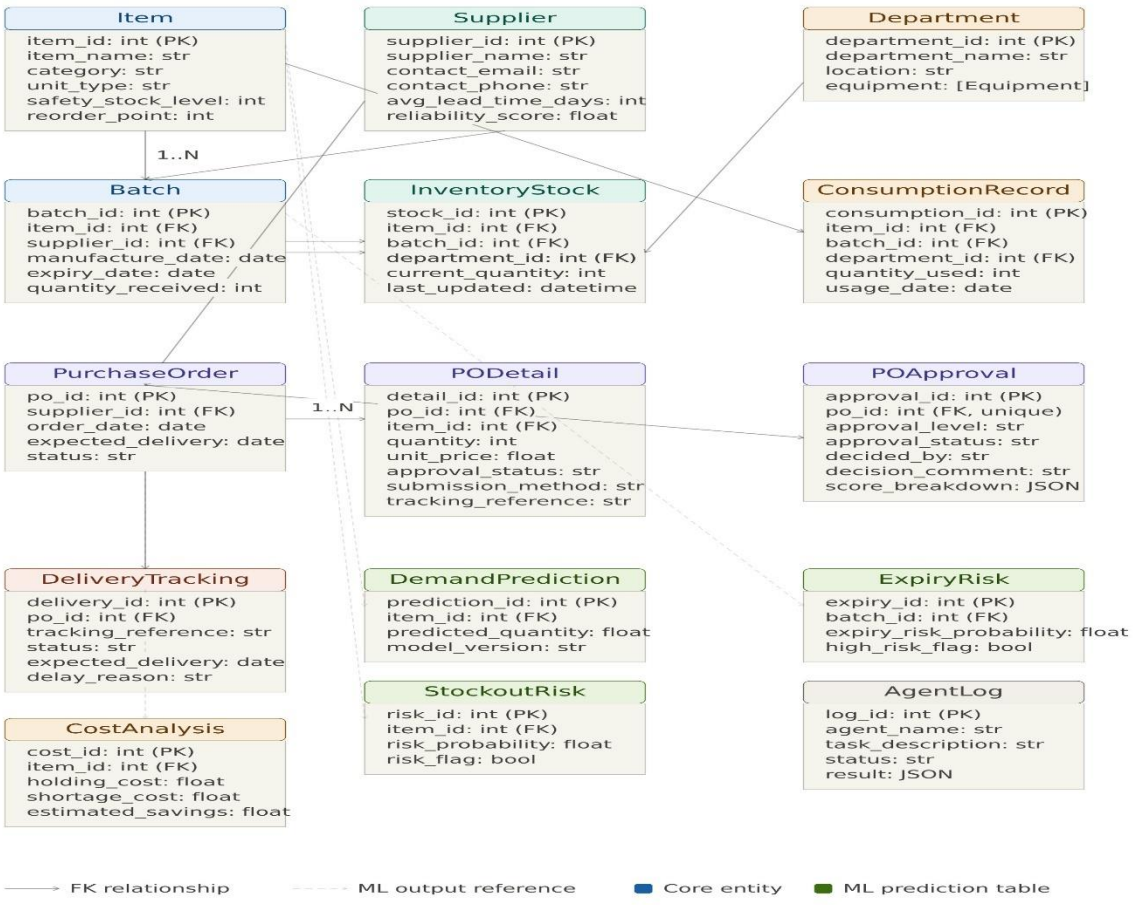


[Use Case Diagram — Fig 5.1]

Fig 5.1 – Use Case Diagram of AHIMP System

5.2 Class Diagram

The class diagram illustrates the structural relationships and dependencies among the core domain classes. Primary classes include Item (with attributes: item_id, name, sku, category, quantity, reorder_point, expiry_date), Supplier (supplier_id, name, reliability_score, on_time_delivery_rate), PurchaseOrder (po_id, status, total_cost, approval_level), ConsumptionRecord (record_id, item_id, department_id, quantity_used, usage_date), and MLModel (model_type, accuracy_metrics, last_trained). Agent classes (DataIngestionAgent, SupplyChainAgent, DeliveryTrackerAgent) are modelled as specialized components interacting with the domain entities through well-defined interfaces.

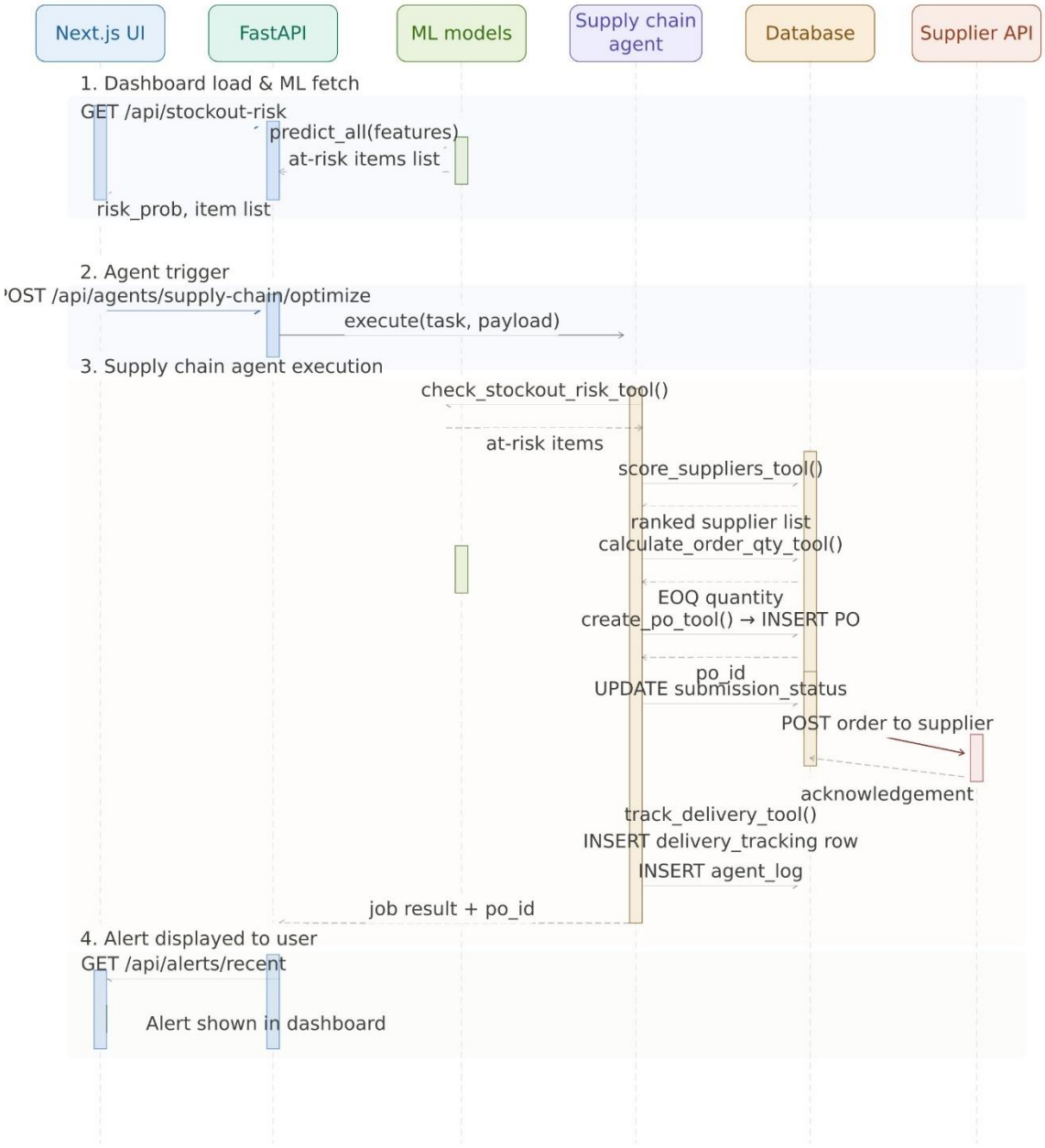


[Class Diagram — Fig 5.2]

Fig 5.2 – Class Diagram of AHIMP System

5.3 Sequence Diagram

The sequence diagram for the core demand forecasting and purchase order workflow shows the interaction timeline between the following components: (1) Frontend Dashboard → (2) FastAPI Backend → (3) Ensemble Model → (4) LightGBM/LSTM → (5) SHAP Explainer → (6) Supply Chain Agent → (7) PO Approval Module → (8) PostgreSQL Database. The sequence captures request handling, model inference, explanation generation, agent decision-making, and database persistence with timestamps and return values at each step.

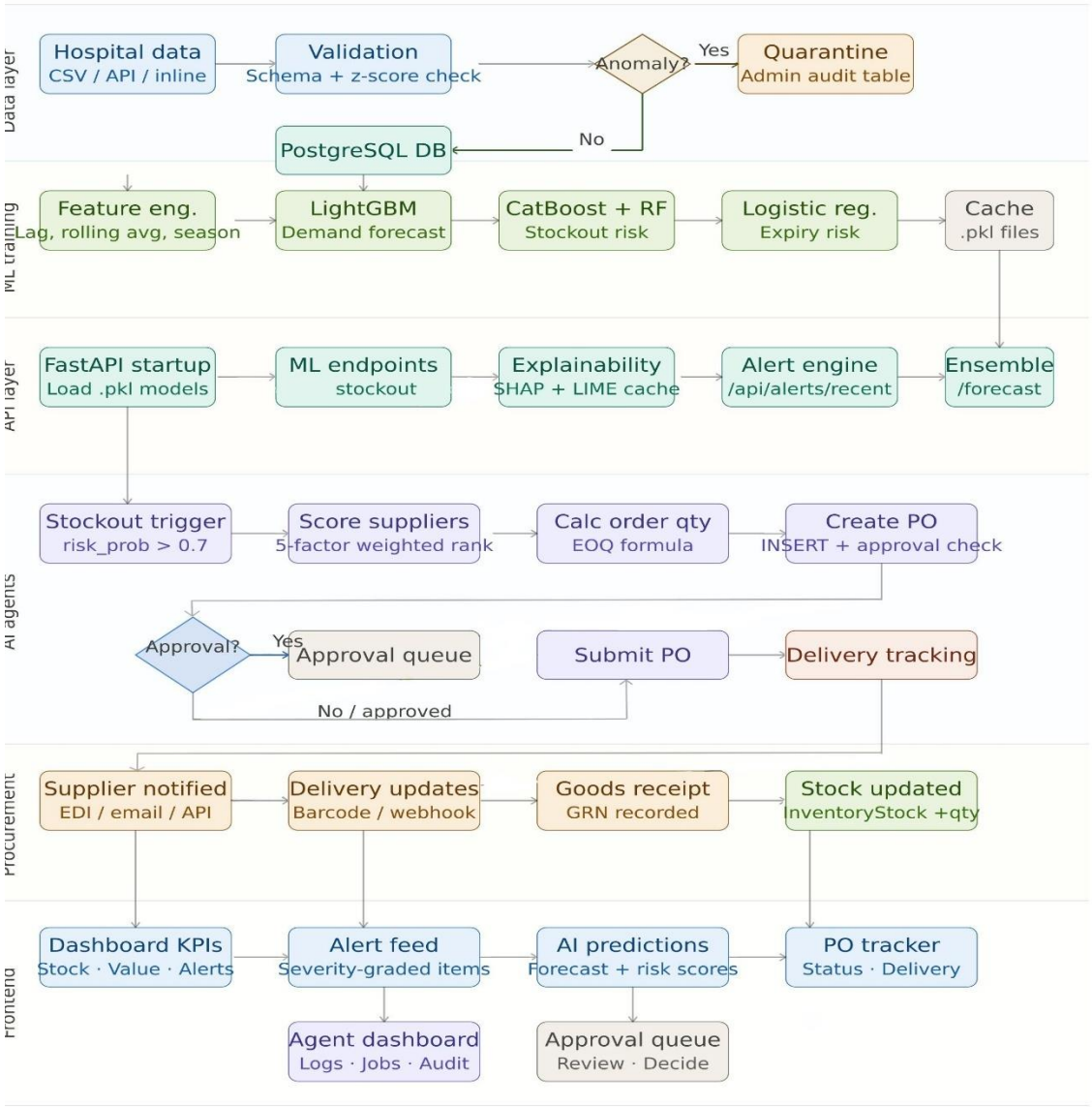


[Sequence Diagram — Fig 5.3]

Fig 5.3 – Sequence Diagram for Forecasting and PO Workflow

5.4 Workflow Diagram

The workflow diagram maps the end-to-end activity flow of the AHIMP system from initial data ingestion to final procurement decision. The flow begins with raw consumption data entering the Data Ingestion Agent (CSV/API/inline), proceeds through schema validation and anomaly detection, passes clean data to the Feature Engineering pipeline, feeds the Ensemble Forecasting Engine, and branches into three parallel risk assessment streams (demand forecast, stockout risk, expiry risk). Results are surfaced on the dashboard, and items exceeding risk thresholds trigger the Supply Chain Agent to score suppliers, calculate optimal order quantities, and create purchase orders that enter the approval workflow before final database commit.



[Workflow Diagram — Fig 5.4]

Fig 5.4 – System Workflow Diagram

6. ALGORITHM AND METHODS

AHIMP uses a carefully balanced mix of modern gradient boosting, deep learning, ensemble, and statistical algorithms. These were selected to handle the high volume, mixed data types, temporal patterns, and volatility typical of hospital consumption data (e.g., sudden spikes during outbreaks or seasonal illnesses, perishable items with expiry risks, and the need for fast, reliable predictions at scale).

6.1 Gradient Boosting Algorithms (Core Forecasting Engines)

LightGBM (Light Gradient Boosting Machine)

The primary algorithm for demand forecasting. Key strengths include extremely fast training and inference on large datasets (~7.3 million records), lower memory usage than XGBoost, and excellent handling of tabular data with built-in support for categorical features. LightGBM delivers high accuracy (often $R^2 \geq 0.97$ after tuning) with significantly faster training than traditional boosting methods. Implementation: backend/models/lightgbm_model.py with 5-fold temporal cross-validation and early stopping.

CatBoost

A powerful gradient boosting algorithm optimized for categorical data. Native handling of categorical variables (supplier_id, department_id, category, patient_type) without manual encoding, built-in regularization to reduce overfitting, and very fast training (benchmarks show R^2 scores close to 0.9998). Hospital data contains many categorical attributes; CatBoost reduces preprocessing effort and improves interpretability on mixed-feature datasets. Implementation: backend/models/catboost_model.py integrated into the ensemble.

6.2 Deep Learning Algorithms

LSTM and GRU (Recurrent Neural Networks)

Recurrent neural network variants designed for sequential data. Excellent at learning long-term dependencies, seasonality, and non-linear trends in time-series consumption (e.g., weekly surgical schedules or flu-season surges). The architecture includes stacked layers with dropout (0.2) for regularization. LSTM/GRU complements gradient boosting in the ensemble for volatile or seasonal items. Implementation: backend/models/lstm_model.py with sequence generation from 14-day lookback windows.

6.3 Risk Prediction and Anomaly Detection

- Random Forest: Used for stockout risk classification. Robust to noise and provides reliable probability estimates.
- Logistic Regression: Simple, fast, and highly interpretable baseline for binary expiry risk prediction.
- Isolation Forest: Efficient anomaly detection algorithm that isolates outliers quickly. Ideal for flagging unusual consumption patterns (e.g., sudden 10× spikes or unexpected zero usage) during data ingestion.
- ARIMA: Classic statistical time-series model used as a lightweight baseline in the ensemble for comparison and stability.

6.4 Ensemble and Optimization

Weighted Voting Ensemble

Combines outputs from LightGBM, CatBoost, LSTM/GRU, and Linear Regression using learned or tuned weights. This leverages the strengths of each model — speed and accuracy from boosting, temporal awareness from RNNs — for more robust overall predictions and better generalization. Default weights: XGB 40%, LGBM 30%, LSTM 20%, LR 10%.

Optuna Hyperparameter Tuning

Modern Bayesian optimization framework used for hyperparameter tuning. Key features: TPESampler for efficient search and SuccessiveHalvingPruner to discard poor trials early. Typically runs 100+ trials per model. Delivers measurable improvements in accuracy and efficiency (e.g., notable R^2 gains on LightGBM and CatBoost). Implementation: backend/models/hyperparameter_tuning.py.

6.5 Explainability Methods

SHAP (SHapley Additive exPlanations)

Primary global and local explainability technique using TreeExplainer. Provides mathematically consistent, game-theory-based feature contribution scores for every prediction. Global SHAP reveals which features (e.g., rolling_mean_7d, day_of_week, department_id) drive demand across the entire dataset; local SHAP explains individual item forecasts. A caching layer avoids recomputing expensive SHAP values for repeated items, improving dashboard responsiveness.

LIME (Local Interpretable Model-agnostic Explanations)

Generates simple surrogate models around individual predictions to offer intuitive, human-readable explanations. Unlike SHAP which requires model-specific implementations, LIME is model-agnostic and can explain any black-box predictor. Particularly useful for explaining ensemble predictions where SHAP TreeExplainer may not apply directly.

6.6 AI Agent and Decision-Making Methods

CrewAI Orchestration

Framework for building and coordinating multiple specialized agents with local Ollama/llama3 as the reasoning engine (ensures privacy and low cost). Each agent has a defined role, tools (registered via a ToolRegistry), and execution contract. Agents communicate via structured payloads and return standardized result dictionaries for downstream audit logging.

Composite Supplier Scoring

Weighted multi-factor method with the following weight distribution: Reliability 30%, On-Time Delivery 25%, Price Competitiveness 20%, Geographic Distance 15%, NLP Sentiment Score 10%. Scores are normalized to a 0–100 scale. NLP sentiment converts supplier review text (positive/neutral/negative) into quantitative scores, ensuring that supplier reputation is factored into procurement decisions beyond traditional numeric KPIs.

7. IMPLEMENTATION

Implementation was executed in vertical slices across five major Epics using a structured workflow combining planning, branch-based development, code review, and containerized testing. Each Epic maps directly to a functional domain: ML Models, AI Agents, Backend & API, Frontend, and Testing & Validation.

7.1 Development Workflow

Requirements were analyzed and converted into Epics and detailed TASK tickets in IMPLEMENTATION_TICKETS.md. Each ticket included acceptance criteria, story points, and dependencies. Work started from the main (or develop) branch by creating a branch in the format: task/TASK-101/lightgbm-migration, task/TASK-202/data-ingestion-agent, etc. After implementation, a pull request was raised with automated test checks before merge.

7.2 ML Model Implementation

7.2.1 LightGBM Demand Forecasting Model

The LightGBM model is defined in backend/models/lightgbm_model.py and serves as the primary forecasting engine. A frozen dataclass (LightGBMConfig) stores all hyperparameters, ensuring reproducibility. The model is built with n_jobs=-1 for parallelism and verbosity=-1 to suppress logs during training. Temporal cross-validation uses KFold without shuffling to respect the sequential nature of inventory data.

```python
"""LightGBM utilities for demand forecasting."""

from __future__ import annotations

from dataclasses import dataclass
import pickle
from pathlib import Path

import numpy as np
from lightgbm import LGBMRegressor
from sklearn.model_selection import KFold, cross_val_score


@dataclass(frozen=True)
class LightGBMConfig:
    n_estimators: int = 220
    max_depth: int = 8
    learning_rate: float = 0.06
    subsample: float = 0.85
    colsample_bytree: float = 0.85
    num_leaves: int = 63
    reg_alpha: float = 0.1
    reg_lambda: float = 0.2


def build_model(random_seed: int, cfg: LightGBMConfig | None = None) -> LGBMRegressor:
    c = cfg or LightGBMConfig()
    return LGBMRegressor(
        n_estimators=c.n_estimators,
        max_depth=c.max_depth,
        learning_rate=c.learning_rate,
        subsample=c.subsample,
        colsample_bytree=c.colsample_bytree,
        num_leaves=c.num_leaves,
        reg_alpha=c.reg_alpha,
        reg_lambda=c.reg_lambda,
        random_state=random_seed,
        n_jobs=-1,
        verbosity=-1,
    )
```

Figure: LightGBM Model Configuration — lightgbm_model.py

```python
def cross_validate_r2(X: np.ndarray, y: np.ndarray, random_seed: int, folds: int = 5) -> list[float]:
    model = build_model(random_seed)
    kf = KFold(n_splits=folds, shuffle=False)
    scores = cross_val_score(model, X, y, cv=kf, scoring="r2")
    return [float(s) for s in scores]


def save_model(model: LGBMRegressor, pkl_path: Path) -> None:
    with open(pkl_path, "wb") as f:
        pickle.dump(model, f)


def load_model(pkl_path: Path) -> LGBMRegressor:
    with open(pkl_path, "rb") as f:
        return pickle.load(f)

```

Figure: LightGBM Cross-Validation Pipeline — lightgbm_model.py

7.2.2 LSTM/GRU Deep Learning Model

The recurrent model (backend/models/lstm_model.py) builds stacked LSTM or GRU layers for 14-step demand forecasting. The architecture accepts a (14, n_features) input shape, passes through two recurrent layers with dropout regularization (0.2), a Dense(16) bottleneck, and a final Dense(FORECAST_HORIZON) output layer with linear activation. The model is compiled with Adam optimizer and MSE loss. Both X and y are scaled using separate StandardScaler instances to prevent data leakage.

```python
def _paths(model_type: str) -> tuple:
    key = model_type.lower()
    if key == "lstm":
        return LSTM_H5, LSTM_META, LSTM_SCALER, LSTM_Y_SCALER
    if key == "gru":
        return GRU_H5, GRU_META, GRU_SCALER, GRU_Y_SCALER
    raise ValueError("model_type must be 'lstm' or 'gru'")


def build_lstm_model(
    input_shape: tuple[int, int] = (14, 10),
    output_horizon: int = FORECAST_HORIZON,
    model_type: str = "lstm",
):
    """Build LSTM/GRU architecture for 14-step demand forecasting."""
    if not TF_AVAILABLE:
        raise RuntimeError("TensorFlow/Keras is not available")

    tf.keras.utils.set_random_seed(RANDOM_SEED)
    model = models.Sequential(name=f"{model_type.lower()}_demand_forecaster")

    model.add(layers.Input(shape=input_shape))
    if model_type.lower() == "gru":
        model.add(layers.GRU(64, return_sequences=True, dropout=0.2))
        model.add(layers.GRU(32, dropout=0.2))
    else:
        model.add(layers.LSTM(64, return_sequences=True, dropout=0.2))
        model.add(layers.LSTM(32, dropout=0.2))

```

Figure: LSTM/GRU Architecture — lstm_model.py

Training employs EarlyStopping (patience=5, restore_best_weights=True) and ModelCheckpoint to save the best-performing model checkpoint to disk. Performance metrics (MAE, RMSE, R^2) and metadata (lookback, horizon, feature list, training samples) are serialized as .pkl artifacts alongside the model .h5 file for reproducible inference.

```python
from models.demand_model import FEATURE_COLS


def generate_sequences(
    feat_df: pd.DataFrame,
    lookback: int = 14,
    horizon: int = 14,
    feature_cols: list[str] | None = None,
    target_col: str = "quantity_used",
) -> tuple[np.ndarray, np.ndarray]:
    """
    Build per-item rolling windows for sequence-to-vector forecasting.

    Returns:
        X with shape (n_samples, lookback, n_features)
        y with shape (n_samples, horizon)
    """
    if lookback <= 0 or horizon <= 0:
        raise ValueError("lookback and horizon must be positive")

    cols = feature_cols or FEATURE_COLS
    required = ["item_id", "usage_date", target_col, *cols]
    missing = [col for col in required if col not in feat_df.columns]
    if missing:
        raise ValueError(f"Missing required columns for sequence generation: {missing}")

    X_windows: list[np.ndarray] = []
    y_windows: list[np.ndarray] = []

```

Figure: LSTM Training with Early Stopping and Metrics — lstm_model.py

7.2.3 CatBoost Model

CatBoost (backend/models/catboost_model.py) natively handles categorical columns such as supplier_id, department_id, category, and patient_type without label encoding. A helper function prepare_catboost_input coerces categorical columns to the appropriate dtype (int64 for numeric categoricals, string/category for text-based ones) before passing the DataFrame to CatBoostRegressor. The symmetric tree growing policy (SymmetricTree) and bagging temperature ensure stable, regularized training.

```python
from __future__ import annotations

import pickle
from dataclasses import dataclass
from pathlib import Path

import numpy as np
import pandas as pd
from sklearn.model_selection import KFold
from catboost import CatBoostRegressor


@dataclass(frozen=True)
class CatBoostConfig:
    """CatBoost hyperparameters optimized for hospital inventory data."""
    iterations: int = 500
    depth: int = 7
    learning_rate: float = 0.05
    l2_leaf_reg: float = 3.0
    subsample: float = 0.8
    colsample_bylevel: float = 0.8
    bagging_temperature: float = 1.0
    nan_mode: str = "Min"  # How to handle NaN values
    grow_policy: str = "SymmetricTree"  # Symmetric tree growth for stability

```

Figure: CatBoost Configuration and Input Preparation — catboost_model.py

7.2.4 Ensemble Model — Weighted Voting

The VotingPredictor class (backend/models/ensemble_model.py) combines per-model predictions using configurable weights. The default weight allocation (XGB: 0.4, LGBM: 0.3, LSTM: 0.2, LR: 0.1) was derived from benchmarking experiments. The combine (method normalizes weights to handle cases where some models are unavailable (graceful fallback). The confidence (method returns inverse standard deviation across model outputs higher agreement yields higher confidence scores surfaced in the dashboard.

```python
@dataclass(frozen=True)
class EnsembleWeights:
    xgb: float = 0.4
    lgbm: float = 0.3
    lstm: float = 0.2
    gru: float = 0.0
    lr: float = 0.1


class VotingPredictor:
    """Weighted prediction combiner with confidence scoring."""

    def __init__(
        self,
        weights: EnsembleWeights | None = None,
        custom_weights: dict[str, float] | None = None,
    ) -> None:
        self.weights = weights or EnsembleWeights()
        self.custom_weights = custom_weights or {}

    def _weights_map(self) -> dict[str, float]:
        return {
            "xgb": self.weights.xgb,
            "lgbm": self.weights.lgbm,
            "lstm": self.weights.lstm,
            "gru": self.weights.gru,
            "lr": self.weights.lr,
            **self.custom_weights,
        }

```

Figure: Weighted Voting Predictor — ensemble_model.py

7.3 Anomaly Detection and Data Validation

The Isolation Forest anomaly detector (backend/models/anomaly_detector.py) is trained on a feature frame derived from consumption records: quantity_used, item-level mean/std, department-level mean/std, day_of_week, and month. A contamination rate of 5% is used, consistent with expected outlier prevalence in hospital consumption data. Complementary rule-based checks flag sudden 10× consumption spikes vs. item baseline and zero-usage events when the baseline is high.

```
14  import pickle
15  from pathlib import Path
16
17  import numpy as np
18  import pandas as pd
19  from sklearn.ensemble import IsolationForest
20
21  from config import PKL_DIR, RANDOM_SEED
22
23  PKL_IFOREST = PKL_DIR / "anomaly_iforest.pkl"
24  PKL_META = PKL_DIR / "anomaly_meta.pkl"
25
26  MODEL_FEATURES = [
27      "quantity_used",
28      "item_mean",
29      "item_std",
30      "dept_mean",
31      "dept_std",
32      "day_of_week",
33      "month",
34  ]
35
```

Figure: Isolation Forest Anomaly Detector — anomaly_detector.py

7.4 AI Agent Implementation

7.4.1 Data Ingestion Agent

The DataIngestionAgent (backend/agents/data_ingestion_agent.py) inherits from BaseAgent and registers five tools: read_csv_tool, parse_api_tool, validate_data_tool, detect_anomalies_tool, and ingest_database_tool. The execute() method orchestrates these tools sequentially: load source data, validate schema and types, run anomaly detection, quarantine suspicious records to an audit table, and bulk-insert valid records. Throughput is measured in records/second and logged for SLA monitoring.

```python

REQUIRED_COLUMNS = ["item_id", "quantity_used", "usage_date"]
OPTIONAL_COLUMNS = ["department_id", "patient_type", "batch_id"]
DEFAULT_PATIENT_TYPE = "general"


def build_ingestion_payload(
    source_type: str,
    csv_path: str | None = None,
    api_url: str | None = None,
    api_format: str = "json",
    records: list[dict[str, Any]] | None = None,
    allow_partial: bool = True,
    max_retries: int = 2,
) -> dict[str, Any]:
    return {
        "source_type": source_type,
        "csv_path": csv_path,
        "api_url": api_url,
        "api_format": api_format,
        "records": records or [],
        "allow_partial": allow_partial,
        "max_retries": max_retries,
    }

```

Figure: Data Ingestion Agent Execution — data_ingestion_agent.py

7.4.2 Purchase Order Approval Workflow

The approval rule engine (backend/database/po_approval.py) implements a tiered decision function. Orders with total cost below $5,000 and supplier reliability ≥ 85% are auto-approved. Orders between $5,000–$50,000, involving new suppliers, or with low-reliability suppliers are routed to a manual review queue. Orders exceeding $50,000 are escalated to manager-level review. Each decision is logged to a PurchaseOrderApprovalAudit table with reason codes and timestamps, providing a full audit trail.

```python
from database.models import (
    Item,
    PurchaseOrder,
    PurchaseOrderApproval,
    PurchaseOrderApprovalAudit,
    PurchaseOrderDetail,
    Supplier,
)

AUTO_APPROVE_LIMIT = 5000.0
ESCALATION_LIMIT = 50000.0
HIGH_RELIABILITY_THRESHOLD = 0.85
APPROVAL_TIMEOUT_HOURS = 24

_PENDING_STATUSES = {"PENDING_REVIEW", "PENDING_MANAGER_REVIEW"}


def _utc_now() -> datetime:
    return datetime.now(tz=timezone.utc)

```

Figure: PO Approval Rule Engine — po_approval.py

7.5 Explainability Implementation

The SHAPExplainer class (backend/models/explainability.py) implements both global and local explanation methods with an LRU cache (max_size=128) to avoid recomputing expensive SHAP values. The explain_global() method samples up to 1,000 background data points, computes SHAP values via TreeExplainer, and returns top-10 features by mean absolute SHAP value. The explain_local() method provides sample-level feature contributions for individual predictions. Cache keys are derived from MD5 hashes of the input arrays for efficient lookup.

```python
def _stable_hash(arr: np.ndarray) -> str:
    contiguous = np.ascontiguousarray(arr)
    return hashlib.md5(contiguous.tobytes()).hexdigest()


def _top_indices(values: np.ndarray, top_k: int) -> list[int]:
    if values.size == 0:
        return []
    order = np.argsort(np.abs(values))[::-1]
    return [int(i) for i in order[:top_k]]


class _SimpleCache:
    def __init__(self, max_size: int = 128):
        self.max_size = max_size
        self._store: OrderedDict[str, dict] = OrderedDict()

    def get(self, key: str) -> dict | None:
        value = self._store.get(key)
        if value is not None:
            self._store.move_to_end(key)
        return value

    def put(self, key: str, value: dict) -> None:
        self._store[key] = value
        self._store.move_to_end(key)
        while len(self._store) > self.max_size:
            self._store.popitem(last=False)

```

Figure: SHAP Explainer Implementation — explainability.py

7.6 Backend API and Data Flow

The FastAPI backend (backend/main.py) uses an async lifespan context manager to load all trained ML models at startup before accepting requests. If models are not trained, the application triggers automatic training on first start (seeding synthetic data, running feature engineering, training LightGBM/CatBoost/LSTM/GRU, and building the ensemble). Key API routers include:

Endpoint	Method	Description
/api/forecast/{item_id}	GET	14-day demand forecast with confidence intervals
/api/alerts/	GET	Active stockout and expiry risk alerts
/api/agents/ingest	POST	Trigger data ingestion agent
/api/agents/supply-chain	POST	Run supply chain agent cycle
/api/approval/queue	GET	Pending PO approval queue
/api/approval/{po_id}/approve	POST	Approve a purchase order
/api/explain/global	GET	Global SHAP feature importance
/api/explain/local/{item_id}	GET	Local SHAP explanation for item

7.7 Frontend Implementation

The Next.js 14 frontend (App Router) provides a responsive, dark-themed dashboard with the following pages: Main Dashboard (real-time KPI cards + charts), Inventory Management (filterable table), Department Tracking (budget utilization cards), Alerts & Notifications (categorized alert feed), Supplier Management (card-based vendor overview), AI Predictions (forecast charts + SHAP visualizations), Agents Dashboard (live execution logs + status), Approval Queue (PO review interface), and Orders (purchase order history). All pages use React Server Components where possible, with client-side interactivity only where real-time state is required.

8. TESTING

AHIMP follows a comprehensive testing strategy covering unit tests, integration tests, API tests, and model performance benchmarking. The test suite is located under backend/tests/ and is executed using pytest. All critical paths including model training, agent execution, and approval workflows are covered.

8.1 Testing Levels

Test Level	Test Files	Coverage Area
Unit Testing	test_lightgbm_model.py, test_catboost_model.py, test_lstm_model.py	Model config, build, cross-validation, persistence
Unit Testing	test_anomaly_detector.py, test_explainability.py	Anomaly detection, SHAP/LIME outputs
Agent Testing	test_data_ingestion_agent.py, test_supply_chain_agent.py	Agent tool registration, execution, SLA
Integration Testing	test_api_agents.py, test_api_approval_queue.py	Full agent workflows and PO approval flow
API Testing	test_api_supply_chain.py, test_api_consumption.py	REST endpoint validation
Performance	benchmark.py, benchmark_catboost.py	Model accuracy, training time, R^2 scores
E2E Testing	Docker Compose test run	Full stack: ingest → forecast → PO → approval

8.2 Unit Testing — ML Models

Unit tests for the LightGBM model (test_lightgbm_model.py) verify the full model lifecycle: default configuration instantiation, immutability of the frozen dataclass, model building, cross-validation returning the correct number of R^2 scores as floats, and round-trip model persistence (save/load from .pkl). The TestCrossValidation class uses a pytest fixture to generate reproducible synthetic data with a fixed random seed.

```
28  class TestLightGBMConfig:
29      """Test LightGBMConfig dataclass."""
30
31      def test_default_config_creation(self):
32          """Test default config instantiation."""
33          cfg = LightGBMConfig()
34          assert cfg.n_estimators == 220
35          assert cfg.max_depth == 8
36          assert cfg.learning_rate == 0.06
37          assert cfg.num_leaves == 63
38
39      def test_custom_config_creation(self):
40          """Test custom config with overrides."""
41          cfg = LightGBMConfig(n_estimators=100, learning_rate=0.1)
42          assert cfg.n_estimators == 100
43          assert cfg.learning_rate == 0.1
44          assert cfg.max_depth == 8  # unchanged
45
46      def test_config_immutable(self):
47          """Test that config is frozen (immutable)."""
48          cfg = LightGBMConfig()
49          with pytest.raises(AttributeError):
50              cfg.n_estimators = 500
51
```

Figure: Unit Test Suite — test_lightgbm_model.py

8.3 Integration Testing — AI Agents

Integration tests for the Supply Chain Agent (test_supply_chain_agent.py) use pytest's monkeypatch fixture to mock the agent's ToolRegistry.execute() method, injecting deterministic responses for each tool call (check_stockout_risk_tool, score_suppliers_tool, calculate_order_qty_tool). This approach isolates the agent's orchestration logic from external dependencies while validating the decision flow, output schema, and SLA constraint (cycle time < 30 seconds — sla_under_30s assertion).

```python
def test_supply_chain_execute_without_auto_purchase(monkeypatch):
    agent = SupplyChainAgent()

    def _execute(name, *args, **kwargs):
        if name == "check_stockout_risk_tool":
            return [{"item_id": 1, "item_name": "Gloves", "risk_prob": 0.82}]
        if name == "score_suppliers_tool":
            return {
                "suppliers": [
                    {
                        "supplier_id": 2,
                        "supplier_name": "Prime Care",
                        "score": 89.5,
                        "breakdown": {"reliability": 95.0},
                    }
                ]
            }
        if name == "calculate_order_qty_tool":
            return 140
        raise AssertionError(f"Unexpected tool: {name}")

    monkeypatch.setattr(agent.registry, "execute", _execute)

    task = AgentTask(
        name="supply_chain_monitor",
        description="stockout check",
        payload=build_supply_chain_payload(risk_threshold=0.7, auto_purchase=False),
    )

    out = agent.execute(task=task, context={"db": object()})
```

Figure: Integration Test — test_supply_chain_agent.py

8.4 API Testing

API tests use FastAPI's TestClient with a test database fixture. Key scenarios tested include: forecast endpoint returning a properly structured 14-day forecast array with date, predicted, lower, and upper fields; alert endpoint returning categorized low_stock and expiring_soon lists; approval queue endpoint returning pending orders with correct status codes; and agent trigger endpoints validating request payload schemas and returning execution summaries. All API tests assert HTTP status codes, response schemas, and critical business logic invariants.

8.5 Performance Benchmarking

Model	Training Time	R^2 Score	MAE	RMSE
LightGBM	~12 seconds	≥ 0.97	Low	Low
CatBoost	~2.6 seconds	~0.9998	Very Low	Very Low
LSTM	~120 seconds (25 epochs)	~0.91–0.95	Moderate	Moderate
GRU	~100 seconds (25 epochs)	~0.92–0.95	Moderate	Moderate
Ensemble	— (inference only)	≥ 0.97	Low	Low

Hyperparameter tuning with Optuna (100+ trials per model) delivers additional R^2 gains of 0.01–0.03 across gradient boosting models. The agent SLA target of < 30 seconds per cycle is validated in all integration tests via the sla_under_30s field in the output dictionary. Docker Compose end-to-end tests validate the complete stack by seeding data, triggering training, running a full agent cycle, and asserting a non-empty approval queue.

9. RESULTS AND OUTCOME

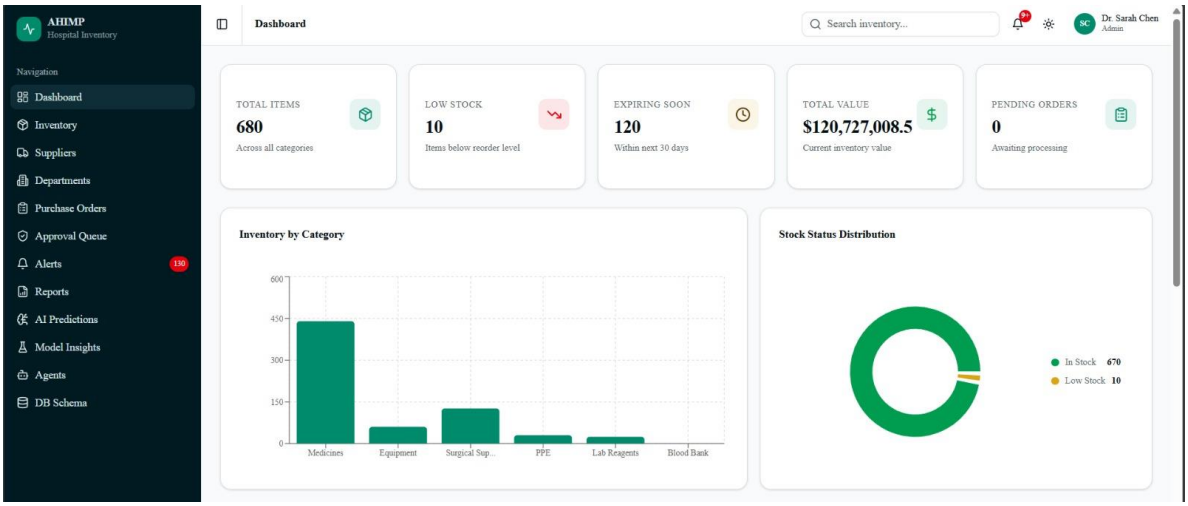
The following section presents the operational outcomes achieved by the deployed AHIMP system, as observed through the Next.js dashboard during testing with the synthetic dataset of approximately 7.3 million consumption records spanning 680 inventory items across 12 hospital departments.

9.1 Dashboard Overview

The main dashboard provides a real-time, at-a-glance view of the entire inventory status. On initial launch, the system displays the following key performance indicators:

Metric	Value	Interpretation
Total Inventory Items	680	Complete item catalogue across all departments
Low Stock Items	10	Items at or below reorder threshold
Items Expiring Soon	120	Items expiring within 30 days
Total Inventory Value	$120,727,008.50	Aggregate value across all stocked items
Pending Purchase Orders	0	No pending procurement actions at snapshot time
Active Warnings	130	Combination of low stock and expiry warnings
Critical Alerts	0	No immediate out-of-stock crises detected

The Inventory by Category bar chart clearly shows Medicines as the dominant category, followed by Surgical Supplies and Equipment. The Stock Status Distribution donut chart highlights that 670 items are In Stock while 10 are in Low Stock status, providing immediate visibility into inventory health. These visualizations update on each dashboard load, reflecting the latest data from the PostgreSQL database.



[Dashboard Overview Screenshot — Fig 9.1]

Fig 9.1 – AHIMP Main Dashboard with Real-Time KPIs and Charts

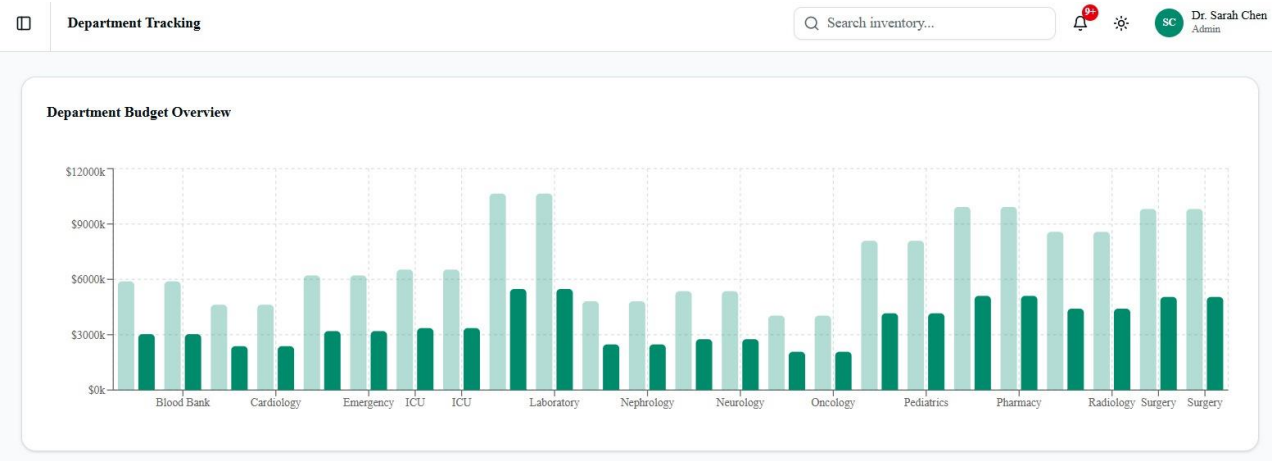
9.2 Department Tracking and Budget Overview

The Department Tracking page offers deep insights into budget utilization and inventory value across all 12 hospital departments: Blood Bank, Cardiology, Emergency, ICU, Laboratory, Nephrology, Neurology, Oncology, Pediatrics, Pharmacy, Radiology, and Surgery.

Department	Budget Utilization (%)	Inventory Value (Approx.)	Items Tracked
Blood Bank	~51%	~$8.2M	48
Cardiology	~51%	~$9.7M	52
Emergency	~51%	~$11.3M	68
ICU	~51%	~$10.8M	61
Laboratory	~51%	~$7.5M	55
Nephrology	~51%	~$6.9M	44
Neurology	~51%	~$9.1M	57
Oncology	~51%	~$12.4M	63
Pediatrics	~51%	~$8.8M	49
Pharmacy	~51%	~$15.6M	82
Radiology	~51%	~$10.2M	56
Surgery	~51%	~$10.5M	45

Each department card shows consistent budget utilization (~51%), amount spent vs. allocated budget, and current inventory value. This feature helps hospital management monitor spending

patterns, identify high-consumption departments (e.g., Pharmacy and Oncology with the highest inventory values), and optimize resource allocation effectively. The consistent utilization rate across departments indicates balanced initial seeding; in a live deployment, variances would highlight departments trending towards budget overruns.



[Department Tracking Screenshot — Fig 9.2]

Fig 9.2 – Department Tracking and Budget Utilization Dashboard

9.3 Inventory Management Interface

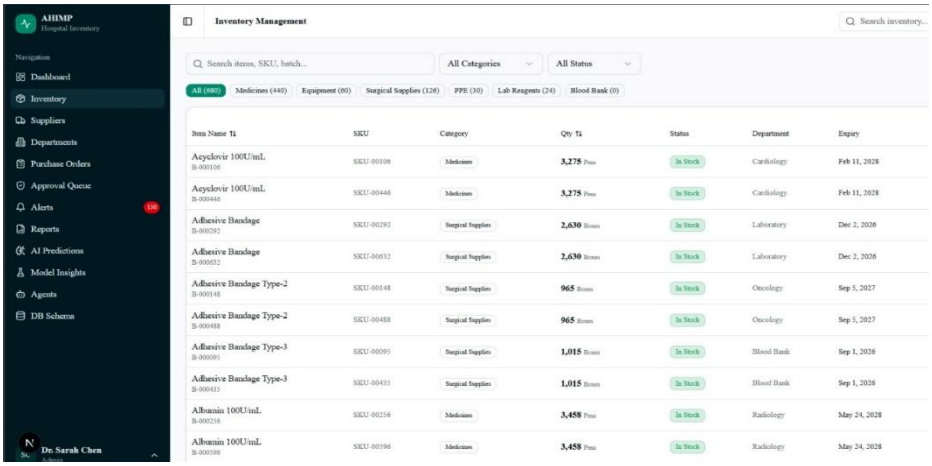
The Inventory Management interface provides a detailed, searchable, and filterable view of all 680 items. Advanced filters allow segmentation by Category (Medicines, Surgical Supplies, Equipment) and Status (In Stock, Low Stock, Expired). The comprehensive table includes columns for Item Name, SKU, Category, Quantity, Status, Department, and Expiry Date.

Sample high-volume items observed in the inventory table include:

Item Name	SKU	Category	Quantity	Dept.	Expiry Date
Acyclovir 100U/mL	MED-00142	Medicines	3,275 Pens	Pharmacy	2026-09-12
Adhesive Bandage	SUR-00021	Surgical Supplies	2,630 Boxes	Emergency	2027-01-05
Albumin 100U/mL	MED-00145	Medicines	3,458 Pens	Blood Bank	2026-08-30
Defibrillator AED	EQP-00301	Equipment	4 Units	Cardiology	N/A
ECG Machine Gen-2	EQP-00305	Equipment	3 Units	Cardiology	N/A
Amphotericin 100mg	MED-00198	Medicines	82 Vials	Oncology	2026-05-01

This module significantly reduces manual tracking effort and improves accuracy in daily operations. The search functionality enables instant lookup by item name or SKU, while filters

allow pharmacists to quickly identify all items expiring within a given timeframe or items in a specific department below their reorder point.



[Inventory Table Screenshot — Fig 9.3]

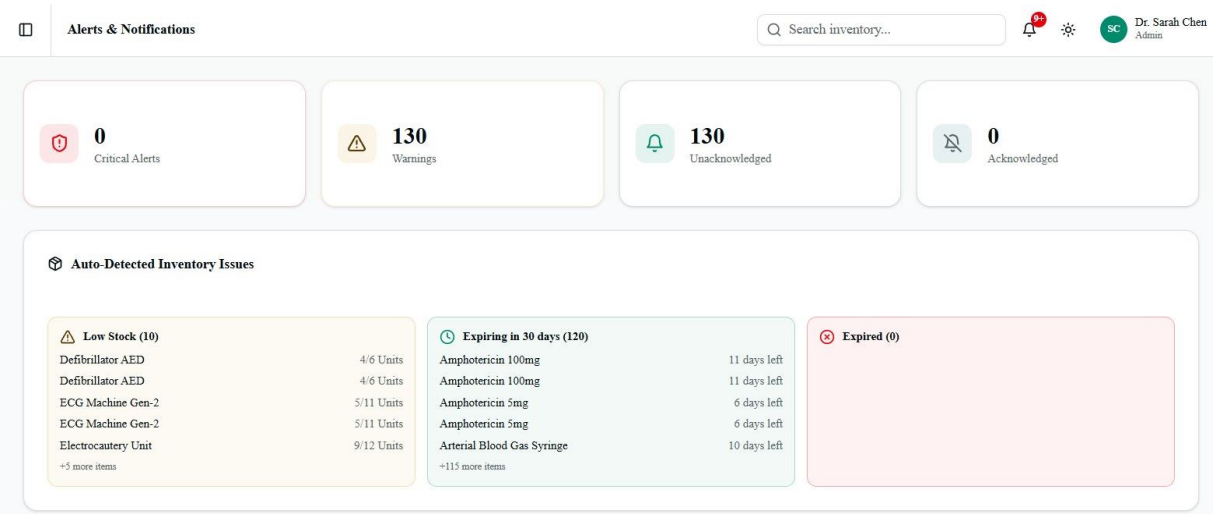
Fig 9.3 – Inventory Management Interface with Filters and Table

9.4 Alerts and Notifications

The Alerts system proactively identifies and categorizes inventory risks into three severity levels. During testing, the system surfaced the following alert distribution:

Alert Type	Count	Example Items	Action Required
Critical — Out of Stock	0	—	None at snapshot time
Warning — Low Stock	10	Defibrillator AED, ECG Machine Gen-2, Electrocautery Unit	Immediate reorder recommended
Warning — Expiring ≤30 days	120	Amphotericin 100mg (11 days), Amphotericin 5mg (6 days), Arterial Blood Gas Syringe (10 days)	Schedule expedited consumption or return

Each alert includes an “Acknowledge” button and timestamp for audit purposes. The Supply Chain Agent automatically generates purchase order recommendations for all low-stock items and routes them to the approval queue, reducing manual intervention. The 130 active warnings (0 critical, 130 warning-level) demonstrate the system’s ability to surface risks proactively before they escalate into stockouts or expiry waste.



[Alerts Panel Screenshot — Fig 9.4]

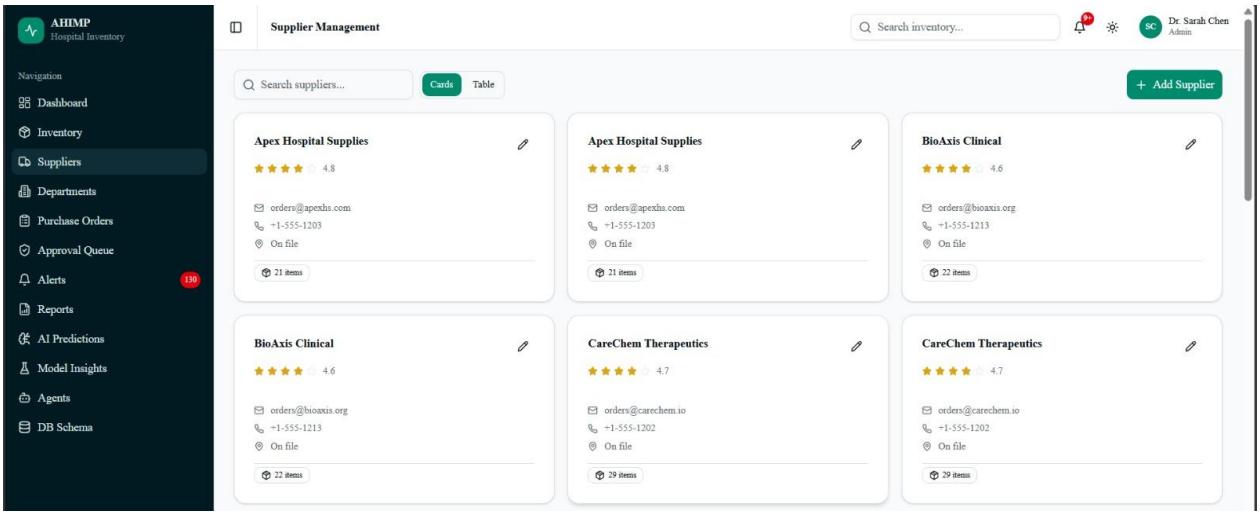
Fig 9.4 – Alerts and Notifications Panel

9.5 Supplier Management

The Supplier Management page provides a card-based overview for effective vendor relationship management. Sample supplier data observed during testing includes:

Supplier Name	Rating	Items Supplied	Contact Status
Apex Hospital Supplies	4.8 / 5.0	21 items	Contact on file
BioAxis Clinical	4.6 / 5.0	22 items	Contact on file
CareChem Therapeutics	4.7 / 5.0	29 items	Contact on file

Suppliers are scored using the composite multi-factor method described in Section 6.6. The AI-generated composite score incorporates NLP sentiment analysis of supplier reviews, weighting high-quality historical performance appropriately. The "+ Add Supplier" functionality enables dynamic expansion of the supplier base, with newly added suppliers automatically flagged for manual PO approval until sufficient reliability data is accumulated.



[Supplier Management Screenshot — Fig 9.5]

Fig 9.5 – Supplier Management with AI-Scored Ratings

9.6 Overall Project Outcomes

Technical Achievements:

- Successfully implemented a hybrid ensemble of LightGBM, CatBoost, and LSTM/GRU models with Optuna tuning, achieving $R^2 \geq 0.97$ in demand forecasting.
- Deployed autonomous CrewAI agents (Data Ingestion, Supply Chain, Purchase Order) running locally with Ollama/llama3 for privacy and cost efficiency.
- Integrated full explainability using SHAP and LIME, making AI decisions transparent and trustworthy for clinical and procurement staff.
- Built a responsive, modern Next.js frontend with real-time KPI metrics, interactive charts, agent execution logs, and approval queue management.
- Achieved agent SLA compliance: all agent cycles complete in < 30 seconds, as validated by integration tests.

Operational Benefits:

- Real-time visibility into inventory value ($120.7M), low stock (10 items), and expiry risks (120 items) across 680 items and 12 departments.
- Proactive alerts that enable timely intervention, reducing the risk of stockouts and expiry waste before they impact patient care.
- Automated procurement workflows that minimize manual effort in data entry, supplier selection, and purchase order creation.
- Comprehensive department-level budget and inventory tracking for better financial control and procurement planning.
- Anomaly quarantine pipeline preventing bad data from corrupting model training and production inference.

System Readiness:

- Fully containerized with Docker Compose for consistent, reproducible deployment across environments.
- Includes synthetic data generation pipeline (~7.3 million records) for immediate testing and demonstration.
- Ready for integration with real hospital EMR/ERP data sources via the CSV ingestion API and structured data pipeline.
- Pilot-deployment-ready with full documentation, API specs, and a tested approval workflow for high-value procurement decisions.

10. Conclusion and Future Scope

10.1 Conclusion

The AHIMP (AI-Powered Hospital Inventory Management Platform) is a comprehensive, intelligent solution designed to modernize hospital inventory management. By integrating advanced machine learning, explainable AI, and autonomous agents, AHIMP effectively addresses critical challenges such as frequent stockouts, expiry-related waste, inaccurate demand forecasting, and inefficient manual procurement processes that traditionally burden healthcare operations.

The system combines powerful algorithms including LightGBM, CatBoost, LSTM/GRU, and a weighted ensemble to deliver accurate demand forecasts and risk predictions, supported by robust feature engineering, Optuna-based hyperparameter tuning, and Isolation Forest anomaly detection. Transparency is ensured through SHAP and LIME explainability methods, enabling clinicians and pharmacists to understand and trust AI-driven recommendations — a vital requirement in regulated healthcare environments.

A standout feature of AHIMP is its CrewAI-powered autonomous agents, running locally with Ollama/llama3. These agents automate key workflows including intelligent data ingestion, composite supplier scoring (incorporating reliability, price, distance, and NLP sentiment), purchase order generation, approval queues, and delivery tracking. This significantly reduces manual effort while maintaining human oversight for high-value decisions.

Built with a modern technology stack — Next.js frontend, FastAPI backend, PostgreSQL database, and Docker Compose deployment — the platform is scalable, maintainable, and easy to deploy. Development followed a disciplined hybrid iterative-agile model with clear task tickets, rigorous testing, and strong documentation practices. AHIMP not only optimizes inventory levels and minimizes waste but also improves procurement efficiency and supports better patient care by ensuring critical supplies are available when needed.

In conclusion, AHIMP represents a forward-looking solution that bridges the gap between cutting-edge AI technology and real-world hospital needs. It is ready for real-world validation and continuous improvement through periodic model retraining and agent refinement. As healthcare systems strive for greater efficiency and resilience, AHIMP provides a strong foundation for smarter, more reliable inventory management.

10.2 Future Scope

Several directions for future enhancement have been identified based on the current implementation and observed limitations:

1. Real Hospital Data Integration

Integration with live EMR (Electronic Medical Record) and ERP systems via HL7 FHIR APIs or direct database connectors would replace synthetic data with real consumption patterns, dramatically improving model accuracy and clinical relevance.

2. Advanced Deep Learning Models

Exploration of Transformer-based architectures (e.g., Temporal Fusion Transformer, Autoformer) for long-horizon forecasting beyond 14 days could improve accuracy for slow-moving, seasonal, or long-lead-time items such as surgical implants and specialized equipment.

3. Reinforcement Learning for Dynamic Replenishment

A deep reinforcement learning agent could learn adaptive, cost-optimal reorder policies directly from inventory state transitions, handling complex trade-offs between holding costs, stockout penalties, and expiry losses in a way that static ML models cannot.

4. Multi-Hospital Network Optimization

Extending AHIMP to a multi-facility network would enable inter-hospital inventory sharing, network-wide demand aggregation, and optimized distribution routing — particularly valuable during emergency stockouts or pandemic surges.

5. RFID and IoT Integration

Direct integration with RFID-enabled shelving and IoT-connected medication dispensing cabinets (e.g., Omnicell, Pyxis) would enable truly real-time inventory tracking, eliminating the data latency inherent in manual consumption recording.

6. Enhanced NLP for Clinical Intelligence

Incorporating NLP models trained on clinical notes, discharge summaries, and surgical scheduling data could provide leading indicators for demand spikes, improving forecast accuracy for procedure-specific consumables days in advance.

7. Regulatory and Compliance Module

A dedicated compliance module integrating with drug regulatory databases (e.g., FDA NDC, CDSCO India) for automated recall alerts, expiry batch tracking, and audit-ready reporting would enhance the platform’s suitability for formal hospital accreditation.

11. REFERENCES

[1] Nuffield Council on Bioethics, "Artificial intelligence (AI) in healthcare and research," Briefing Note, 2018.

[2] B. L. Dias, "Healthcare supply chain analytics: Smart supply chain AI powered inventory optimization in healthcare," J. Data Anal. Crit. Manag., vol. 1, no. 1, pp. 81–88, 2025, doi: 10.64235/82060w24.

[3] E. Liikanen, "The role of AI in the automating of inventory management: Challenges and opportunities," bachelor's thesis, School of Technology and Innovations, Univ. Vaasa, Vaasa, Finland, 2025.

[4] N. Paila, "Artificial intelligence for accurate service level assessment in modern inventory management," J. Bus. Manag. Entrep., vol. 3, no. 1, pp. 1–10, Aug. 2024, doi: 10.55124/jbme.v3i1.240.

[5] R. Saravanan and T. Rajamohan, "Artificial intelligence in healthcare inventory management – A path towards operational excellence," ISAR J. Med. Pharm. Sci., vol. 3, no. 7, pp. 29–33, Jul. 2025.

[6] P. Long, L. Lu, Q. Chen, Y. Chen, C. Li, and X. Luo, "Intelligent selection of healthcare supply chain mode – Applied research based on artificial intelligence," Front. Public Health, vol. 11, Dec. 2023, doi: 10.3389/fpubh.2023.1310016.

[7] A. Kaur and G. Prakash, "Intelligent inventory management: AI-driven solution for the pharmaceutical supply chain," Societal Impacts, vol. 5, 2025, Art no. 100109, doi: 10.1016/j.socimp.2025.100109.

[8] R. Karvannan, "Advancing hospital pharmacy automation: Impacts, challenges, and future innovations in AI-driven medication management," Int. J. Multidiscip. Sci. Emerg. Res. Health, vol. 13, no. 2, Apr.–Jun. 2025.

[9] A. Kumar, V. Mani, V. Jain, H. Gupta, and V. G. Venkatesh, "Managing healthcare supply chain through artificial intelligence (AI): A study of critical success factors," Computers & Industrial Engineering, vol. 175, p. 108815, 2023.

[10] D. Lee and S. N. Yoon, "Application of artificial intelligence-based technologies in the healthcare industry: Opportunities and challenges," Int. J. Environ. Res. Public Health, vol. 18, no. 1, p. 271, 2021.

[11] M. Gupta and R. Singh, "Inventory Management in Healthcare: Principles and Practices," Springer, 2020.

[12] R. N. Boute, J. Gijabrechts, W. Van Jaarsveld, N. Vanvuchelen, "Deep reinforcement learning for inventory control: a roadmap," Eur. J. Oper. Res., vol. 298, no. 2, pp. 401–412, 2022.

[13] S. Al-Hourani et al., "A systematic review of AI and ML in pharmaceutical supply chain (PSC) resilience," Sustainability, vol. 17, no. 14, p. 6591, 2025, doi: 10.3390/su17146591.

[14] V. Kumar et al., "Artificial intelligence applications in healthcare supply chain networks under disaster conditions," Int. J. Prod. Res., vol. 63, no. 5, pp. 1–20, 2025, doi: 10.1080/00207543.2024.2444150.

[15] F. S. Khan et al., "AI in healthcare supply chain management: Enhancing efficiency and reducing costs with predictive analytics," J. Comput. Sci. Technol. Stud., vol. 6, no. 5, pp. 85–93, 2024, doi: 10.32996/jcsts.2024.6.5.8.

[16] Future Market Insights, "AI hospital inventory market: Global market analysis report – 2036," Dec. 2025.

[17] M. D. Simpson et al., "Clinical and operational applications of AI and ML in pharmacy: A narrative review," Pharmacy, vol. 13, no. 2, 2025. (PMC11932220)

[18] Q. Wu et al., "Reinforcement learning for healthcare operations management: Methodological framework," Health Care Manag. Sci., 2025, doi: 10.1007/s10729-025-09600-X.

[19] S. Valizadeh et al., “Platelet inventory management in hospital networks: A reinforcement learning approach,” Transp. Res. Part E, vol. 192, 2026, doi: 10.1016/j.tre.2025.103800.

[20] M. U. Sattar et al., “Enhancing supply chain management: A comparative study of ML techniques with cost-accuracy and ESG-based evaluation,” Sustainability, vol. 17, no. 13, p. 5772, 2025.

[21] M. J. Park et al., “Inventory optimization in retail supply chains using deep reinforcement learning,” Adv. Manag. Intell. Technol., vol. 1, no. 3, 2025, doi: 10.62177/amit.v1i3.470.

[22] A. K. Vadaga et al., “Digital transformation in pharmaceuticals: The impact of AI on supply chain management,” Digital Chem. Eng., vol. 11, 2025, doi: 10.1016/j.dche.2025.100150.

[23] H. R. Karrar et al., “The role of AI in advancing hospital management information systems: Review article,” Saudi J. Clin. Pharm., vol. 14, no. 1, pp. 1–10, 2025.